

TechnobizTrader

MetaTrader 5 Integration Guide

Version: 2.0 | Date: May 3, 2026 | Status: Production Ready

TechnobizTrader — MetaTrader 5 Integration Guide

Complete Step-by-Step Connection & Configuration

Table of Contents

- 1. [Overview](#overview)
- 2. [Prerequisites](#prerequisites)
- 3. [Part 1: MT5 Setup & Account Configuration](#part-1-mt5-setup--account-configuration)
- 4. [Part 2: Python Environment & SDK Installation](#part-2-python-environment--sdk-installation)
- 5. [Part 3: API Configuration in TechnobizTrader](#part-3-api-configuration-in-technobiztrader)
- 6. [Part 4: Connection Testing & Validation](#part-4-connection-testing--validation)
- 7. [Part 5: Market Data Integration](#part-5-market-data-integration)
- 8. [Part 6: Trade Execution Setup](#part-6-trade-execution-setup)
- 9. [Part 7: Deployment & Production](#part-7-deployment--production)
- 10. [Troubleshooting](#troubleshooting)
- 11. [Security Best Practices](#security-best-practices)

Overview

MetaTrader 5 (MT5) serves as the **market data provider** and **trade execution engine** for TechnobizTrader. This integration enables:

- **Real-time OHLC data** (Daily, 4H, 1H candles)
- **Live market data streaming** for Trend-Master analysis
- **Trade execution** for Trader-Master signals
- **Position monitoring** and order management
- **Account balance tracking** and risk calculation
- **Trade history** logging for performance analysis

[Code Block - See markdown file for details]

TechnobizTrader Agents

Market Data Provider Interface (abstract)

MT5 Provider (mt5_provider.py)

MT5 Python SDK (MetaTrader5 library)

MT5 Terminal ↔ Broker Servers

Live Market Data & Trade Execution

[Code Block - See markdown file for details]

Prerequisites

- **Operating System:** Windows 7 or later (MT5 desktop runs on Windows)
- **Linux/Mac users:** Use Windows VM or cloud-hosted MT5 terminal

- **Python:** 3.10 or higher
- **RAM:** Minimum 4GB (recommended 8GB for stable operation)
- **Network:** Stable internet connection (minimum 5 Mbps)
- **MT5 Terminal:** Latest version installed and running
- **Active MT5 Trading Account** with a broker supporting:
 - Forex (EURUSD, GBPUSD, etc.)
 - Real-time data API access
 - Automated trading (Expert Advisors via Python SDK)
 - Minimum account balance: \$100-\$500 (depends on position sizing)
- **Broker Support:** Most major brokers offer MT5 (IG, Saxo Bank, Pepperstone, IC Markets, etc.)
- **Demo Account:** Recommended for testing before live trading

[Table - See markdown file for details]

[Table - See markdown file for details]

[Table - See markdown file for details]

[Table - See markdown file for details]

[Table - See markdown file for details]

[Table - See markdown file for details]

Part 1: MT5 Setup & Account Configuration

1. **Visit your broker's website** (e.g., IG.com, Saxo Bank)
2. **Download MT5 Terminal** for Windows
3. **Run the installer** and follow prompts
 - Default installation path: `C:\Program Files\TerminalName\`
4. **Launch MT5 Terminal**
5. **Log in with your trading account credentials**
 - Account number
 - Password
 - Select appropriate server
1. **Open MT5 Terminal**
2. **Navigate to:** Tools → Options
3. **Go to "Expert Advisors" tab**
4. **Enable the following:**
 - ✓ Allow automated trading
 - ✓ Allow live trading
 - ✓ Allow DLL imports (if scripts need it)
5. **Click OK** to save settings
1. **In MT5 Terminal, click:** View → Account Information
2. **Record the following:**
 - **Account Number:** (e.g., 123456)
 - **Account Name:** (used for login)
 - **Server Name:** (e.g., "IG.MID-Demo" or "IBroker-Live")
 - **Account Balance:** (starting capital)
 - **Account Leverage:** (e.g., 1:30)
3. **Keep these credentials secure** — you'll need them for Python configuration

1. **In MT5, right-click Market Watch**
 2. **Select:** Symbols
 3. **Search for and add:**
 - EURUSD (major forex pair)
 - GBPUSD, USDJPY (alternatives)
 - Any symbols your trading strategy targets
 4. **Ensure all symbols have:**
 - ✓ Active quotes (green checkmark)
 - ✓ Bid/Ask spread visible
 - ✓ Real-time tick data
1. **In MT5, navigate:** Tools → News Calendar
 2. **Verify** that economic events are visible (for risk management)
 3. **Enable notifications** for high-impact events (optional but recommended)

Part 2: Python Environment & SDK Installation

Open Command Prompt or PowerShell in your project directory:

[Code Block - See markdown file for details]

Navigate to Trading_Agent directory

```
cd c:\Users\erick\Downloads\Trading_Agent
```

Create virtual environment

```
python -m venv venv
```

Activate virtual environment

For Command Prompt:

```
venv\Scripts\activate
```

For PowerShell:

venv\Scripts\Activate.ps1

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

Ensure pip is up to date

```
pip install --upgrade pip
```

Install MetaTrader5 SDK

```
pip install MetaTrader5
```

Install additional dependencies

```
pip install numpy pandas
```

Verify installation

```
python -c "import MetaTrader5 as mt5; print(mt5.__version__)"
```

[Code Block - See markdown file for details]

Create a test file `test\_mt5\_connection.py`:

[Code Block - See markdown file for details]

```
import MetaTrader5 as mt5
```

```
import sys
```

```
print("[TEST] Connecting to MT5 Terminal...")
```

Initialize MT5 connection

```
if mt5.initialize():
    print("[SUCCESS] MT5 connection established")
    print(f" Version: {mt5.version()}")
    print(f" Terminal: {mt5.terminal_info()}")
    print(f" Account: {mt5.account_info()}")
    mt5.shutdown()
else:
    print("[ERROR] Failed to connect to MT5")
    print(f" Ensure MT5 Terminal is running")
    sys.exit(1)
```

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

```
python test_mt5_connection.py
```

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

```
[TEST] Connecting to MT5 Terminal...
```

```
[SUCCESS] MT5 connection established
```

```
Version: (5, 0, 37)
```

```
Terminal: AccountInfo(login=123456, server='IG.MID-Demo', ...)
```

```
Account: TerminalInfo(company='Company', name='Terminal', ...)
```

[Code Block - See markdown file for details]

- ✓ Ensure MT5 Terminal is **running and logged in**
- ✓ Check firewall: MT5 needs local port access
- ✓ Restart MT5 and try again

Part 3: API Configuration in TechnobizTrader

Create `.env` in the project root:

[Code Block - See markdown file for details]

■■■ MT5 CONNECTION



MT5_ENABLED=true
MT5_TERMINAL_TIMEOUT=5000 # milliseconds

From Step 1.3 "Account Details"

MT5_LOGIN=123456

MT5_PASSWORD=your_password_here

MT5_SERVER=IG.MID-Demo # or your broker server name

■■■ TRADING PARAMETERS



TRADING_SYMBOL=EURUSD
TRADING_LEVERAGE=1:30
MAX_POSITION_SIZE=2.0 # lots
RISK_PER_TRADE=0.02 # 2% of account

■■■ ACCOUNT PROTECTION



MAX_CONCURRENT_TRADES=3
MAX_DAILY_DRAWDOWN=0.05 # 5%
MIN_ACCOUNT_BALANCE=100 # USD

■■■ API LOGGING

DEBUG_MODE=false

LOG_LEVEL=INFO

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

.env

.env.local

secrets.py

config/secrets.py

[Code Block - See markdown file for details]

Edit `config/api\_config.py`:

[Code Block - See markdown file for details]

import os

from dotenv import load_dotenv

Load environment variables from .env

```
load_dotenv()
```



MT5 CONFIGURATION



```
MT5_CONFIG = {  
    'enabled': os.getenv('MT5_ENABLED', 'false').lower() == 'true',  
    'terminal_timeout': int(os.getenv('MT5_TERMINAL_TIMEOUT', '5000')),  
    'credentials': {  
        'login': int(os.getenv('MT5_LOGIN', 0)),  
        'password': os.getenv('MT5_PASSWORD', ""),  
        'server': os.getenv('MT5_SERVER', ""),  
        'reconnect_attempts': 3,  
        'reconnect_delay': 2, # seconds
```



TRADING PARAMETERS



```
TRADING_CONFIG = {
'symbols': [os.getenv('TRADING_SYMBOL', 'EURUSD')],
'leverage': os.getenv('TRADING_LEVERAGE', '1:30'),
'max_position_size': float(os.getenv('MAX_POSITION_SIZE', '2.0')),
'risk_per_trade': float(os.getenv('RISK_PER_TRADE', '0.02')),
'max_concurrent_trades': int(os.getenv('MAX_CONCURRENT_TRADES', '3')),
'max_daily_drawdown': float(os.getenv('MAX_DAILY_DRAWDOWN', '0.05')),
```



DATA TIMEFRAMES



```
TIMEFRAMES = {  
    'daily': 'D1',  
    '4h': 'H4',  
    '1h': 'H1',
```

Minimum candles required for analysis

```
MIN_CANDLES = {
'daily': 20, # ~20 days
'4h': 20, # ~3 days
'1h': 50, # ~2 days
def get_mt5_config():
    """Get MT5 configuration with validation"""
    if not MT5_CONFIG['credentials']['login']:
        raise ValueError("MT5_LOGIN not configured in .env")
    if not MT5_CONFIG['credentials']['password']:
        raise ValueError("MT5_PASSWORD not configured in .env")
    if not MT5_CONFIG['credentials']['server']:
        raise ValueError("MT5_SERVER not configured in .env")
    return MT5_CONFIG
def get_trading_config():
    """Get trading configuration"""
    return TRADING_CONFIG
[Code Block - See markdown file for details]
Update the MT5 data provider implementation:
[Code Block - See markdown file for details]
import MetaTrader5 as mt5
import logging
from datetime import datetime, timedelta, timezone
from typing import List, Dict, Optional
from config.api_config import get_mt5_config, TIMEFRAMES, MIN_CANDLES
from market_data.models import Candle
logger = logging.getLogger(__name__)
class MT5Provider:
    """MetaTrader 5 market data provider"""
    def __init__(self, config=None):
        self.config = config or get_mt5_config()
        self.connected = False
        self.terminal_info = None
        self.account_info = None
    def connect(self):
        """Establish connection to MT5 Terminal"""
        try:
            logger.info("[MT5] Initializing connection...")
```

Initialize MT5

```
if not mt5.initialize():  
    raise ConnectionError("Failed to initialize MT5")
```

Store terminal info

```
self.terminal_info = mt5.terminal_info()
```

```
logger.info(f"[MT5] Terminal: {self.terminal_info.name} v{self.terminal_info.build}")
```


Get account info

```
self.account_info = mt5.account_info()
if not self.account_info:
    raise ConnectionError("Failed to get account info")
logger.info(f"[MT5] Account: {self.account_info.login} | "
f"Balance: ${self.account_info.balance:.2f} | "
f"Equity: ${self.account_info.equity:.2f}")
self.connected = True
return True
except Exception as e:
    logger.error(f"[MT5] Connection failed: {e}")
    return False
def disconnect(self):
    """Close MT5 connection"""
    if self.connected:
        mt5.shutdown()
        self.connected = False
    logger.info("[MT5] Disconnected")
def get_ohlc(self, symbol: str, timeframe: str, bars: int = 100) -> List[Candle]:
    """
    Fetch OHLC data from MT5
    Args:
    symbol: Trading pair (e.g., 'EURUSD')
    timeframe: Timeframe code ('D1', 'H4', 'H1')
    bars: Number of bars to fetch
    Returns:
    List of Candle objects
    """
    try:
        if not self.connected:
            logger.error("[MT5] Not connected")
            return []
```

Convert timeframe

```
tf_map = {  
    'daily': mt5.TIMEFRAME_D1,  
    'D1': mt5.TIMEFRAME_D1,  
    '4h': mt5.TIMEFRAME_H4,  
    'H4': mt5.TIMEFRAME_H4,  
    '1h': mt5.TIMEFRAME_H1,  
    'H1': mt5.TIMEFRAME_H1,  
    mt5_timeframe = tf_map.get(timeframe, mt5.TIMEFRAME_H1)
```

Request candles

```
rates = mt5.copy_rates_from_pos(symbol, mt5_timeframe, 0, bars)
if rates is None or len(rates) == 0:
    logger.warning(f"[MT5] No data for {symbol} {timeframe}")
return []
```

Convert to Candle objects

```
candles = []
for rate in rates:
    candle = Candle(
        timestamp=datetime.fromtimestamp(rate['time'], tz=timezone.utc),
        open=rate['open'],
        high=rate['high'],
        low=rate['low'],
        close=rate['close'],
        volume=int(rate['tick_volume']),
        symbol=symbol,
        timeframe=timeframe,
        candles.append(candle)
    logger.info(f"[MT5] Fetched {len(candles)} candles for {symbol} {timeframe}")
    return candles
except Exception as e:
    logger.error(f"[MT5] OHLC fetch failed: {e}")
    return []

def place_order(self, symbol: str, direction: str, volume: float,
    entry: float, stop_loss: float, take_profit: float) -> Optional[int]:
    """
    Place trade order on MT5
    Args:
    symbol: Trading pair
    direction: 'BUY' or 'SELL'
    volume: Position size in lots
    entry: Entry price
    stop_loss: Stop loss level
    take_profit: Take profit level
    Returns:
    Order ticket number or None on failure
    """
    try:
        if not self.connected:
            logger.error("[MT5] Not connected")
            return None
```

Determine order type

```
order_type = mt5.ORDER_TYPE_BUY if direction == 'BUY' else mt5.ORDER_TYPE_SELL
```

Create order

```
request = {  
    'action': mt5.TRADE_ACTION_DEAL,  
    'symbol': symbol,  
    'volume': volume,  
    'type': order_type,  
    'price': entry,  
    'sl': stop_loss,  
    'tp': take_profit,  
    'comment': 'TechnobizTrader',  
    'type_time': mt5.ORDER_TIME_GTC,  
    'type_filling': mt5.ORDER_FILLING_IOC,  
}
```

Send order

```
result = mt5.order_send(request)
if result.retcode != mt5.TRADE_RETCODE_DONE:
    logger.error(f"[MT5] Order failed: {result.comment}")
    return None
logger.info(f"[MT5] Order placed: {direction} {volume}L {symbol} @ {entry}")
return result.order
except Exception as e:
    logger.error(f"[MT5] Order placement failed: {e}")
    return None
def get_account_balance(self) -> float:
    """Get current account balance"""
    if not self.connected:
        return 0.0
    account = mt5.account_info()
    return account.balance if account else 0.0
def get_positions(self) -> List[Dict]:
    """Get open positions"""
    if not self.connected:
        return []
    positions = mt5.positions_get()
    return [
        'ticket': p.ticket,
        'symbol': p.symbol,
        'type': 'BUY' if p.type == mt5.POSITION_TYPE_BUY else 'SELL',
        'volume': p.volume,
        'entry_price': p.price_open,
        'current_price': p.price_current,
        'profit': p.profit,
        for p in positions
```

[Code Block - See markdown file for details]

Part 4: Connection Testing & Validation

Create `test\_mt5\_integration.py`:

[Code Block - See markdown file for details]

```
import sys
import logging
from market_data.mt5_provider import MT5Provider
from config.api_config import get_mt5_config, TRADING_CONFIG
logging.basicConfig(level=logging.INFO, format='%(levelname)s] %(message)s')
logger = logging.getLogger(__name__)
def test_connection():
    """Test MT5 connection"""
```

```

logger.info("=" * 60)
logger.info("TEST 1: MT5 Terminal Connection")
logger.info("=" * 60)
provider = MT5Provider()
if provider.connect():
    logger.info("✓ Connection successful")
    logger.info(f" Terminal: {provider.terminal_info.name}")
    logger.info(f" Account: {provider.account_info.login}")
    logger.info(f" Balance: ${provider.account_info.balance:.2f}")
    logger.info(f" Equity: ${provider.account_info.equity:.2f}")
    provider.disconnect()
    return True
else:
    logger.error("✗ Connection failed")
    return False

def test_data_fetch():
    """Test OHLC data fetching"""
    logger.info("\n" + "=" * 60)
    logger.info("TEST 2: OHLC Data Fetching")
    logger.info("=" * 60)
    provider = MT5Provider()
    if not provider.connect():
        logger.error("✗ Cannot connect to MT5")
        return False
    symbol = TRADING_CONFIG['symbols'][0]
    timeframes = ['D1', 'H4', 'H1']
    all_good = True
    for tf in timeframes:
        candles = provider.get_ohlc(symbol, tf, bars=10)
        if not candles:
            logger.error(f"✗ No data for {symbol} {tf}")
            all_good = False
        else:
            latest = candles[-1]
            logger.info(f"✓ {symbol} {tf}: {len(candles)} candles")
            logger.info(f" Latest: O={latest.open:.5f} H={latest.high:.5f} "
                f"L={latest.low:.5f} C={latest.close:.5f}")
            provider.disconnect()
    return all_good

def test_account_info():
    """Test account info retrieval"""
    logger.info("\n" + "=" * 60)
    logger.info("TEST 3: Account Information")
    logger.info("=" * 60)

```



```
provider = MT5Provider()
if not provider.connect():
    logger.error("✗ Cannot connect to MT5")
    return False

balance = provider.get_account_balance()
positions = provider.get_positions()
logger.info(f"✓ Account Balance: ${balance:.2f}")
logger.info(f"✓ Open Positions: {len(positions)}")

for pos in positions:
    logger.info(f" - {pos['symbol']} {pos['type']} {pos['volume']}L @ "
               f"${pos['entry_price']:.5f} (P&L:; ${pos['profit']:.2f})")

provider.disconnect()

return True

def main():
    """Run all tests"""
    logger.info("\n████████████████████████████████████████████████████████████████████████████████")
    logger.info("██ TechnobizTrader — MT5 Integration Test Suite ██")
    logger.info("████████████████████████████████████████████████████████████████████████████████\n")

    tests = [
        ("Connection", test_connection),
        ("Data Fetch", test_data_fetch),
        ("Account Info", test_account_info),
    ]

    results = []

    for name, test_func in tests:
        try:
            results.append((name, test_func()))
        except Exception as e:
            logger.error(f"✗ {name} test failed with exception: {e}")
            results.append((name, False))
```

Summary

```
logger.info("\n" + "=" * 60)
logger.info("TEST SUMMARY")
logger.info("=" * 60)
for name, passed in results:
    status = "PASS" if passed else "FAIL"
    logger.info(f"{{name:<20}} : {{status}}")
all_passed = all(r[1] for r in results)
logger.info("=" * 60)
logger.info(f"Overall: {{'ALL TESTS PASSED ✓' if all_passed else 'SOME TESTS FAILED ✗'}}")
return 0 if all_passed else 1
if __name__ == '__main__':
    sys.exit(main())
```

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

Activate virtual environment (if not already active)

```
venv\Scripts\activate
```

Run tests

```
python test_mt5_integration.py
```

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

[illegible]

■ TechnobizTrader — MT5 Integration Test Suite ■

[REDACTED]

=====

TEST 1: MT5 Terminal Connection

✓ Connection successful

Terminal: MetaTrader 5

Account: 123456

Balance: \$1000.00

Equity: \$1000.00

=====

TEST 2: OHLC Data Fetching

=====

✓ EURUSD D1: 10 candles

Latest: O=1.0950 H=1.1000 L=1.0900 C=1.0980

✓ EURUSD H4: 10 candles

✓ EURUSD H1: 10 candles

=====

TEST 3: Account Information

=====

✓ Account Balance: \$1000.00

✓ Open Positions: 0

=====

TEST SUMMARY

=====

Connection : PASS

Data Fetch : PASS

Account Info : PASS

=====

Overall: ALL TESTS PASSED ✓

[Code Block - See markdown file for details]

Part 5: Market Data Integration

Edit `market\_data/cache.py`:

[Code Block - See markdown file for details]

```
from datetime import datetime, timedelta, timezone
```

from typing import Dict, List

```
import json
```

```

class DataCache:
    """Market data cache with TTL"""
    def __init__(self, ttl_minutes: int = 5):
        self.cache = {}
        self.ttl = timedelta(minutes=ttl_minutes)
    def set(self, key: str, value: List):
        """Cache data with timestamp"""
        self.cache[key] = {
            'data': value,
            'timestamp': datetime.now(timezone.utc)
        }
    def get(self, key: str) -> List:
        """Get cached data if not expired"""
        if key not in self.cache:
            return None
        entry = self.cache[key]
        if datetime.now(timezone.utc) - entry['timestamp'] > self.ttl:
            del self.cache[key]
            return None
        return entry['data']
    def clear(self):
        """Clear all cache"""
        self.cache.clear()

```

[Code Block - See markdown file for details]

Create/update `main.py`:

[Code Block - See markdown file for details]

```

import asyncio
import logging

from datetime import datetime

from market_data.mt5_provider import MT5Provider
from agents.trend_master.trend_master import TrendMaster
from agents.analyse_master.analyse_master import AnalyseMaster
from agents.trader_master.trader_master import TraderMaster
from config.api_config import get_mt5_config, TRADING_CONFIG

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(name)s — %(message)s'
)
logger = logging.getLogger(__name__)

class TradingAgency:
    """Main trading agency orchestrator"""
    def __init__(self):
        self.mt5_provider = MT5Provider()
        self.trend_master = TrendMaster()
        self.analyse_master = AnalyseMaster()
        self.trader_master = TraderMaster()

```

```
self.running = False
async def initialize(self):
    """Initialize all components"""
    logger.info("■" * 60)
    logger.info("TechnobizTrader Initialization")
    logger.info("■" * 60)
```

Connect to MT5

```
logger.info("[INIT] Connecting to MT5...")
if not self.mt5_provider.connect():
    logger.error("[INIT] Failed to connect to MT5")
    return False
logger.info("[INIT] Initialization complete ✓")
return True

async def shutdown(self):
    """Shutdown all components"""
    logger.info("[SHUTDOWN] Disconnecting from MT5...")
    self.mt5_provider.disconnect()
    logger.info("[SHUTDOWN] Complete")
    async def run_trading_cycle(self):
        """Execute one complete trading cycle"""
        symbol = TRADING_CONFIG['symbols'][0]
        logger.info(f"[CYCLE] Starting analysis for {symbol}...")
```

Fetch multi-timeframe data

```
daily = self.mt5_provider.get_ohlc(symbol, 'D1', bars=20)
h4 = self.mt5_provider.get_ohlc(symbol, 'H4', bars=20)
h1 = self.mt5_provider.get_ohlc(symbol, 'H1', bars=50)
if not (daily and h4 and h1):
    logger.warning("[CYCLE] Insufficient data")
return
```


Trend analysis

```
trend_report = await self.trend_master.analyze({
'daily': daily,
'4h': h4,
'1h': h1,
if not trend_report:
logger.warning("[CYCLE] No trend identified")
return
logger.info(f"[CYCLE] Trend: {trend_report.bias} (conf: {trend_report.confidence}%")
```

Entry signal analysis

```
trade_signal = await self.analyse_master.analyze(  
trend_report.to_dict(),  
candle_data={'1h': h1},  
symbol=symbol  
if not trade_signal:  
logger.info("[CYCLE] No trade signal generated")  
return  
logger.info(f"[CYCLE] Signal: {trade_signal.direction} @ {trade_signal.entry_level}")
```

Trade execution

```
execution = await self.trader_master.analyze(trade_signal.to_dict())
if not execution:
    logger.info("[CYCLE] Signal rejected by risk management")
    return
logger.info(f"[CYCLE] ✓ Trade executed: {execution.to_dict()}")
async def run(self):
    """Main trading loop"""
    if not await self.initialize():
        return
    self.running = True
    cycle_count = 0
    try:
        while self.running:
            cycle_count += 1
            logger.info(f"\n■■ CYCLE {cycle_count} ({datetime.now().strftime('%H:%M:%S')}) ■■")
            await self.run_trading_cycle()
            logger.info("■ Waiting 4 hours for next analysis...")
            await asyncio.sleep(3600 * 4) # 4-hour interval
        except KeyboardInterrupt:
            logger.info("\n[USER] Shutdown requested")
        except Exception as e:
            logger.error(f"[ERROR] {e}", exc_info=True)
    finally:
        await self.shutdown()
    async def main():
        agency = TradingAgency()
        await agency.run()
    if __name__ == '__main__':
        asyncio.run(main())
```

[Code Block - See markdown file for details]

Part 6: Trade Execution Setup

Add to `agents/trader\_master/risk\_manager.py`:

[Code Block - See markdown file for details]

```
import logging
from config.api_config import TRADING_CONFIG
logger = logging.getLogger(__name__)
class RiskManager:
    """Risk management and position sizing"""
    @staticmethod
    def calculate_position_size(account_balance: float, entry: float,
                               stop_loss: float, symbol: str = 'EURUSD') -> float:
```

"""

Calculate position size based on 2% risk rule

Formula:

Max Risk \$ = Account Balance × 2%

Risk Pips = |Entry - SL| / 0.0001

Position Lots = Max Risk \$ / (Risk Pips × Pip Value)

Args:

account_balance: Current account balance

entry: Entry price

stop_loss: Stop loss level

symbol: Trading symbol

Returns:

Position size in lots

"""

```
max_risk_percent = TRADING_CONFIG['risk_per_trade']
```

```
max_risk_dollars = account_balance * max_risk_percent
```

Calculate risk in pips (assuming 4-decimal pairs)

```
risk_pips = abs(entry - stop_loss) / 0.0001
```

Pip value for EURUSD = \$0.01 per pip per standard lot

`pip_value_per_lot = 10.0 # $10 per 0.1 pips for standard lot`

Position size calculation

$$\text{position_size} = \text{max_risk_dollars} / (\text{risk_pips} * 0.1)$$

Apply limits

```
min_size = 0.01 # micro lot
max_size = TRADING_CONFIG['max_position_size']
position_size = max(min_size, min(position_size, max_size))
logger.info(f"[POSITION] Account: ${account_balance:.2f}")
logger.info(f"[POSITION] Risk: {max_risk_percent*100}% = ${max_risk_dollars:.2f}")
logger.info(f"[POSITION] Stop Loss: {risk_pips:.1f} pips")
logger.info(f"[POSITION] Size: {position_size:.2f} lots")
return position_size

@staticmethod
def validate_execution(trade_signal: Dict, current_positions: int,
account_balance: float) -> bool:
    """
    Validate trade execution against risk constraints

    Checks:
    • Confidence >= 75%
    • Risk/Reward >= 1:2
    • Concurrent trades < 3
    • Daily drawdown < 5%
    """
    errors = []
```


Check confidence

```
if trade_signal['confidence'] < 75:  
    errors.append(f"Confidence {trade_signal['confidence']}% < 75%")
```

Check R:R

```
if trade_signal['risk_reward_ratio'] < 2.0:  
    errors.append(f"R:R {trade_signal['risk_reward_ratio']:.2f} < 1:2")
```

Check concurrent trades

```
if current_positions >= TRADING_CONFIG['max_concurrent_trades']:
    errors.append(f"Already {current_positions} open trades (max 3)")
if errors:
    logger.warning("[VALIDATION] Execution rejected:")
    for error in errors:
        logger.warning(f" - {error}")
    return False
logger.info("[VALIDATION] ✓ All checks passed")
return True
```

[Code Block - See markdown file for details]

Part 7: Deployment & Production

Create `config/production.env`:

[Code Block - See markdown file for details]

■■■ PRODUCTION MT5 SETTINGS



MT5_ENABLED=true

MT5_TERMINAL_TIMEOUT=10000

MT5_LOGIN=YOUR_LIVE_ACCOUNT_ID

MT5_PASSWORD=YOUR_LIVE_PASSWORD

MT5_SERVER=YOUR_BROKER_LIVE_SERVER

■■■ PRODUCTION TRADING



TRADING_SYMBOL=EURUSD

TRADING_LEVERAGE=1:30

MAX_POSITION_SIZE=1.0 # More conservative for live

RISK_PER_TRADE=0.01 # 1% per trade in production

■■■ STRICTER LIMITS



MAX_CONCURRENT_TRADES=2
MAX_DAILY_DRAWDOWN=0.03 # 3% in production
MIN_ACCOUNT_BALANCE=1000 # Require \$1000 minimum

■■■ LOGGING



DEBUG_MODE=false

LOG_LEVEL=WARNING # Less verbose in production

[Code Block - See markdown file for details]

Create `deploy.sh`:

[Code Block - See markdown file for details]

TechnobizTrader Deployment Script

[illegible]

```
echo "■ TechnobizTrader — Production Deployment ■"
```

[illegible]

Check Python version

```
python_version=$(python --version 2>&1 | awk '{print $2}')  
echo "[CHECK] Python version: $python_version"
```

Check MT5 Terminal

```
echo "[CHECK] Verifying MT5 Terminal..."  
python test_mt5_connection.py  
if [ $? -ne 0 ]; then  
echo "[ERROR] MT5 Terminal not accessible"  
exit 1
```

Backup current .env

```
if [ -f ".env" ]; then
cp .env ".env.backup.$(date +%s)"
echo "[BACKUP] Created .env backup"
```

Copy production config

```
cp config/production.env .env
```

```
echo "[CONFIG] Loaded production settings"
```

Install dependencies

```
echo "[INSTALL] Installing/updating dependencies..."  
pip install -r requirements.txt
```

Run final tests

```
echo "[TEST] Running pre-deployment tests..."
python test_mt5_integration.py
if [ $? -ne 0 ]; then
echo "[ERROR] Tests failed"
exit 1
echo "[SUCCESS] Deployment ready"
echo ""
echo "Next steps:"
echo " 1. Review .env configuration"
echo " 2. Run: python main.py"
[Code Block - See markdown file for details]
Create `utils/logger.py`:
[Code Block - See markdown file for details]
import logging
import logging.handlers
from datetime import datetime
def setup_logger(name: str, log_file: str = None) -> logging.Logger:
    """Configure logger with file and console handlers"""
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)
```

Format

```
formatter = logging.Formatter(  
    '%(asctime)s [%(levelname)s] %(name)s — %(message)s',  
    datefmt='%Y-%m-%d %H:%M:%S'
```

Console handler

```
console = logging.StreamHandler()
console.setLevel(logging.INFO)
console.setFormatter(formatter)
logger.addHandler(console)
```


File handler (if specified)

```
if log_file:
    file_handler = logging.handlers.RotatingFileHandler(
        log_file,
        maxBytes=10*1024*1024, # 10MB
        backupCount=5
    )
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(formatter)
    logger.addHandler(file_handler)
    return logger
```

Initialize main logger

```
main_logger = setup_logger(  
'TechnobizTrader',  
log_file=f"logs/trading_{datetime.now().strftime('%Y%m%d')}.log"  
[Code Block - See markdown file for details]
```

Troubleshooting

1. Open MT5 Terminal on your desktop
2. Ensure you're logged in with trading account
3. Wait 5 seconds for initialization
4. Try connecting again

[Code Block - See markdown file for details]

```
python test_mt5_connection.py
```

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

Create .env from template

```
cp .env.template .env
```

Edit with your MT5 credentials

notepad .env

[Code Block - See markdown file for details]

[Code Block - See markdown file for details]

pip install MetaTrader5

python -c "import MetaTrader5; print(MetaTrader5.__version__)"

[Code Block - See markdown file for details]

1. In MT5 Terminal: Tools → Options → Account Information
2. Verify login number and server name
3. Update `.env` with correct values
4. Restart MT5 and Python script

1. In MT5: Right-click Market Watch → Symbols
2. Find and add symbol (e.g., EURUSD)
3. Verify green checkmark appears
4. Wait 10 seconds for quotes to populate

Security Best Practices

- ✓ Never commit `.env` to git (add to `.gitignore`)
- ✓ Use strong passwords (20+ characters, mixed case/numbers/symbols)
- ✓ Rotate credentials every 90 days
- ✓ Use separate accounts: Demo for testing, Live for trading
- ✓ Ensure firewall allows MT5 (port 443 default)
- ✓ Use VPN for remote connections
- ✓ Disable UPnP on MT5 terminal
- ✓ Run on trusted network only
- ✓ Enable two-factor authentication with broker
- ✓ Restrict API access to specific IPs (if broker offers)
- ✓ Use API keys with minimal permissions
- ✓ Log all API calls for audit trail
- ✓ Encrypt sensitive files (credentials, trade history)
- ✓ Back up database regularly to secure location
- ✓ Monitor unusual trading patterns
- ✓ Set account alerts with broker
- ✓ Keep Python and all packages updated
- ✓ Run system on isolated VM for production
- ✓ Monitor system resources (CPU, RAM, disk)
- ✓ Implement circuit breaker (auto-shutdown if errors persist)
- ✓ Maintain manual override capability

Verification Checklist

Before going live:

- [] MT5 Terminal installed and login working
- [] `.env` configured with correct credentials
- [] Virtual environment created and activated

- ☐ All dependencies installed (``pip install -r requirements.txt``)
- ☐ ``test_mt5_integration.py`` passes all 3 tests
- ☐ Multi-timeframe data fetching works
- ☐ Order placement tested on demo account
- ☐ Position sizing calculations verified
- ☐ Risk management constraints enforced
- ☐ Logging and audit trail configured
- ☐ Monitoring alerts set up
- ☐ Backup and recovery procedures tested
- ☐ Manual shutdown procedure documented

Quick Start Commands

[Code Block - See markdown file for details]

Initial setup

```
cd c:\Users\erick\Downloads\Trading_Agent  
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt
```

Testing

`python test_mt5_connection.py`

`python test_mt5_integration.py`

Production deployment

```
bash deploy.sh
```

```
python main.py
```


View logs

```
tail -f logs/trading_20260503.log
```

[Code Block - See markdown file for details]

Support & Resources

- **MT5 Documentation:** help.metrader5.com
- **Python SDK GitHub:** github.com/louisnow/MetaTrader5
- **Broker Support:** Contact your broker's technical support
- **TechnobizTrader Issues:** Check project GitHub issues